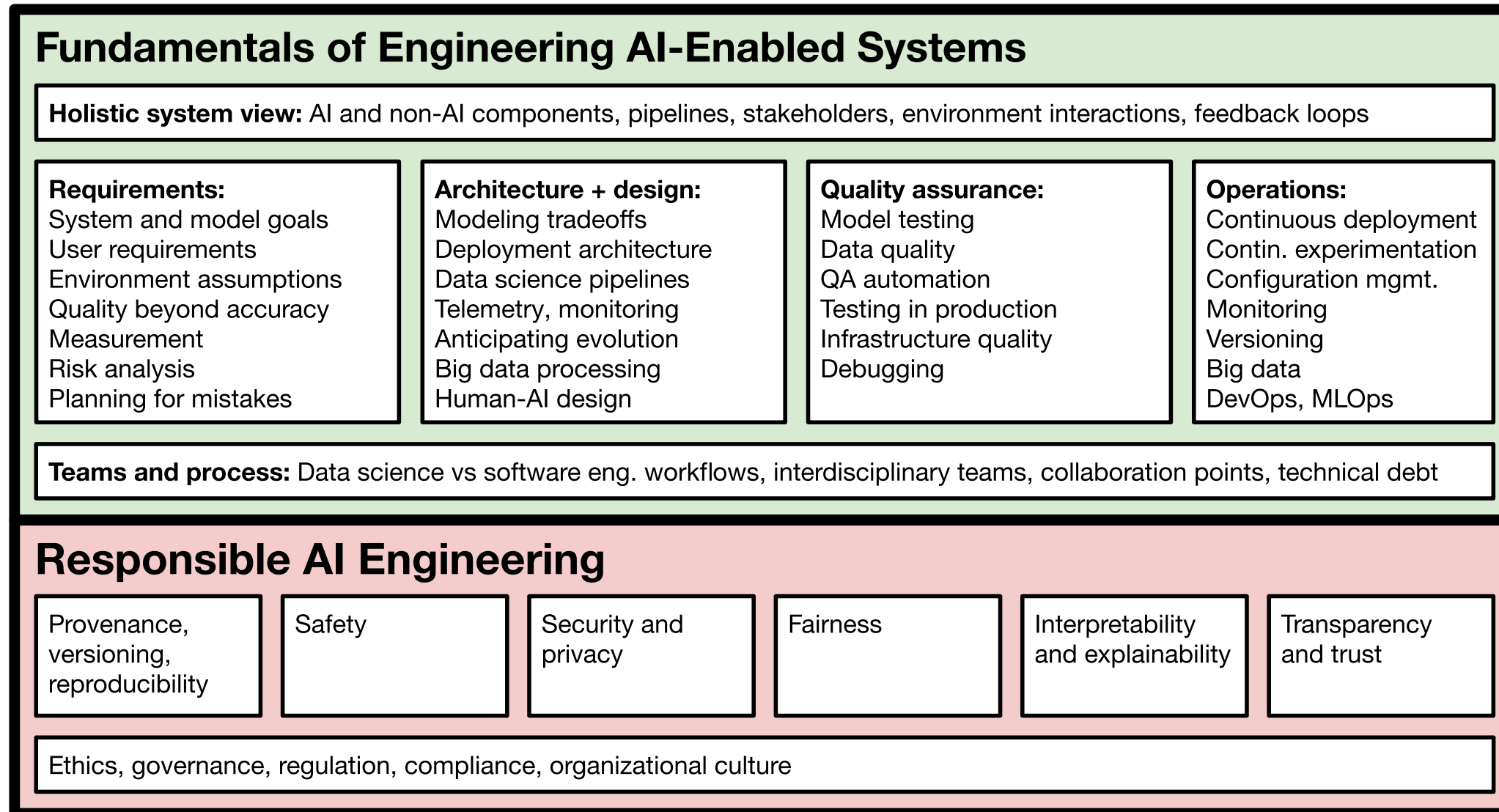




Machine Learning in Production

Deploying a Model

Deeper into architecture and design...



Learning Goals

- Understand important quality considerations when deploying ML components
- Follow a design process to explicitly reason about alternative designs and their quality tradeoffs
- Gather data to make informed decisions about what ML technique to use and where and how to deploy it
- Understand the power of design patterns for codifying design knowledge

- Create architectural models to reason about relevant characteristics
- Critique the decision of where an AI model lives (e.g., cloud vs edge vs hybrid), considering the relevant tradeoffs
- Deploy models locally and to the cloud
- Document model inference services

Readings

Required reading:

- Hulten, Geoff. "[Building Intelligent Systems: A Guide to Machine Learning Engineering](#)." Apress, 2018, Chapter 13 (Where Intelligence Lives).
-  Daniel Smith. "[Exploring Development Patterns in Data Science](#)." TheoryLane Blog Post. 2017.

Recommended reading:

- Rick Kazman, Paul Clements, and Len Bass. [Software architecture in practice](#). Addison-Wesley Professional, 2012, Chapter 1

Deploying a Model is Easy

Deploying a Model is Easy

Model inference component as function/library

```
from sklearn.linear_model import LogisticRegression
model = ... # learn model or load serialized model ...
def infer(feature1, feature2):
    return model.predict(np.array([[feature1, feature2]]))
```

Deploying a Model is Easy

Model inference component as a service

```
from flask import Flask, escape, request
app = Flask(__name__)
app.config['UPLOAD_FOLDER'] = '/tmp/uploads'
detector_model = ... # load model...

# inference API that returns JSON with classes
# found in an image
@app.route('/get_objects', methods=['POST'])
def pred():
    uploaded_img = request.files["images"]
    converted_img = ... # feature encoding of uploaded img
```

Deploying a Model is Easy

Packaging a model inference service in a container

```
FROM python:3.8-buster
RUN pip install uwsgi==2.0.20
RUN pip install tensorflow==2.7.0
RUN pip install flask==2.0.2
RUN pip install gunicorn==20.1.0
COPY models/model.pf /model/
COPY ./serve.py /app/main.py
WORKDIR ./app
EXPOSE 4040
CMD ["gunicorn", "-b 0.0.0.0:4040", "main:app"]
```

Deploying a Model is Easy

Model inference component as a service in the cloud

- Package in container or other infrastructure
- Deploy in cloud infrastructure
- Auto-scaling with demand ("*Stateless Serving Functions Pattern*")
- MLOps infrastructure to automate all of this (more on this later)
 - [BentoML](#) (low code service creation, deployment, model registry),
 - [Cortex](#) (automated deployment and scaling of models on AWS),
 - [TFX model serving](#) (tensorflow GRPC services)
 - [Seldon Core](#) (no-code model service and many many additional services for monitoring and operations on Kubernetes)

But is it really easy?

Offline use?

Deployment at scale?

Hardware needs and operating cost?

Frequent updates?

Integration of the model into a system?

Meeting system requirements?

Every system is different!

Every System is Different

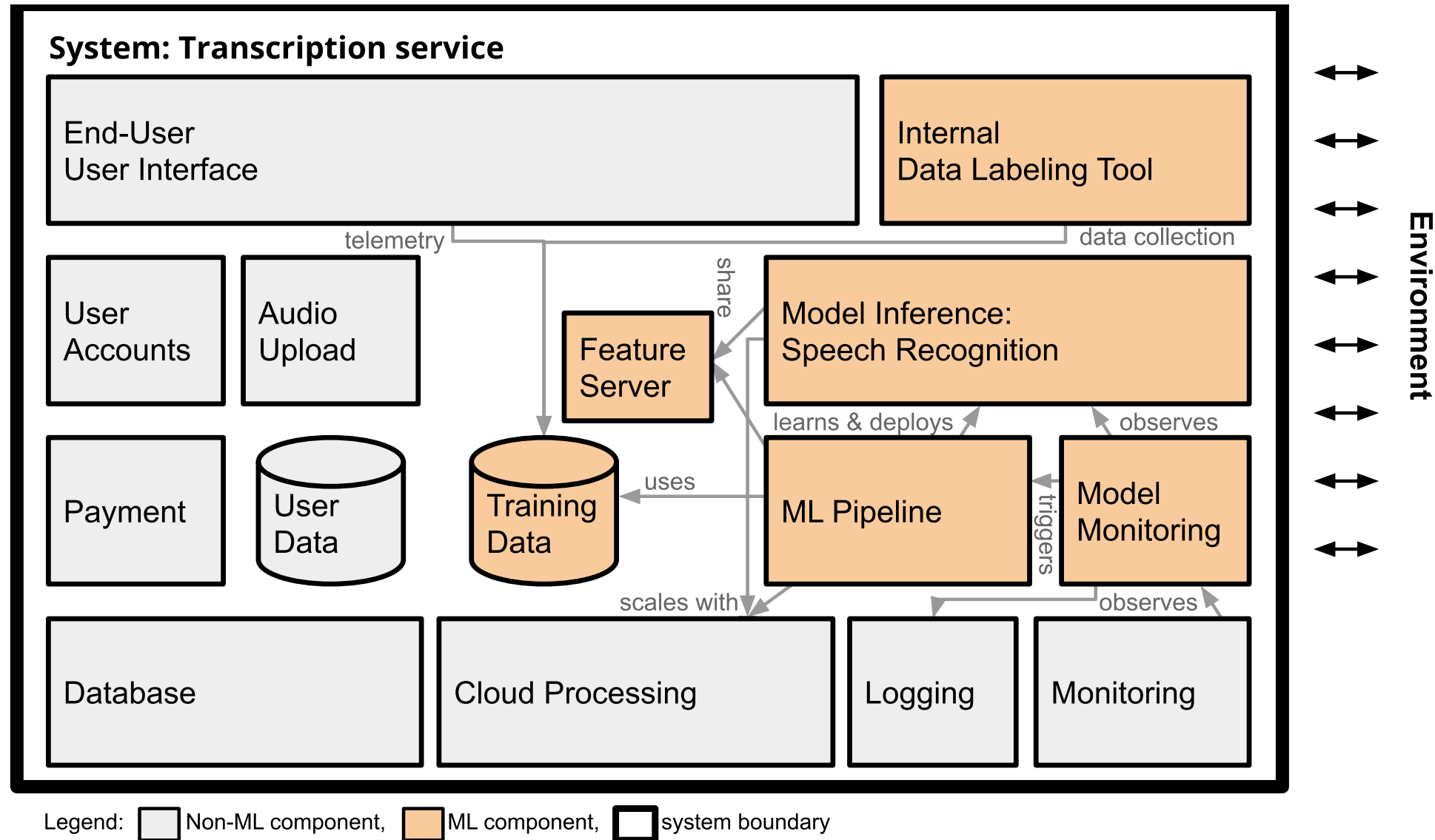
Personalized music recommendations for Spotify

Transcription service startup

Self-driving car

Smart keyboard for mobile device

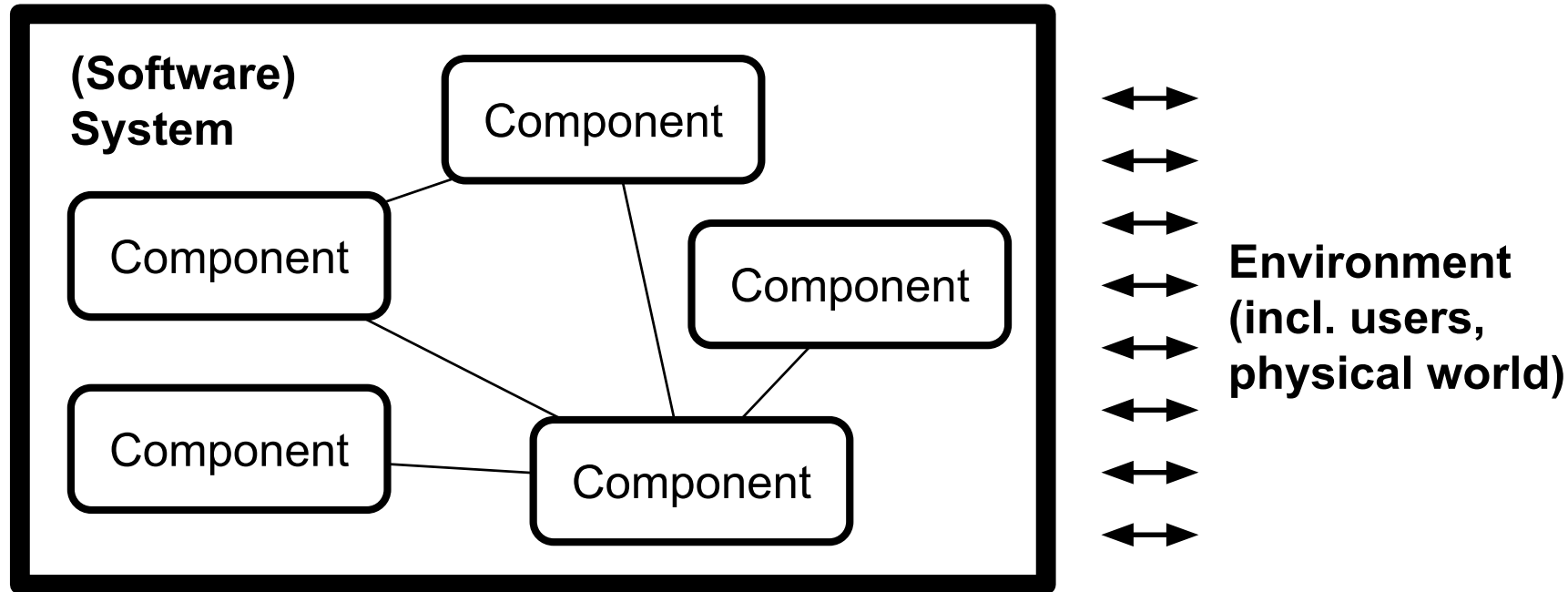
Inference is a Component within a System



Recall: Thinking like a Software Architect

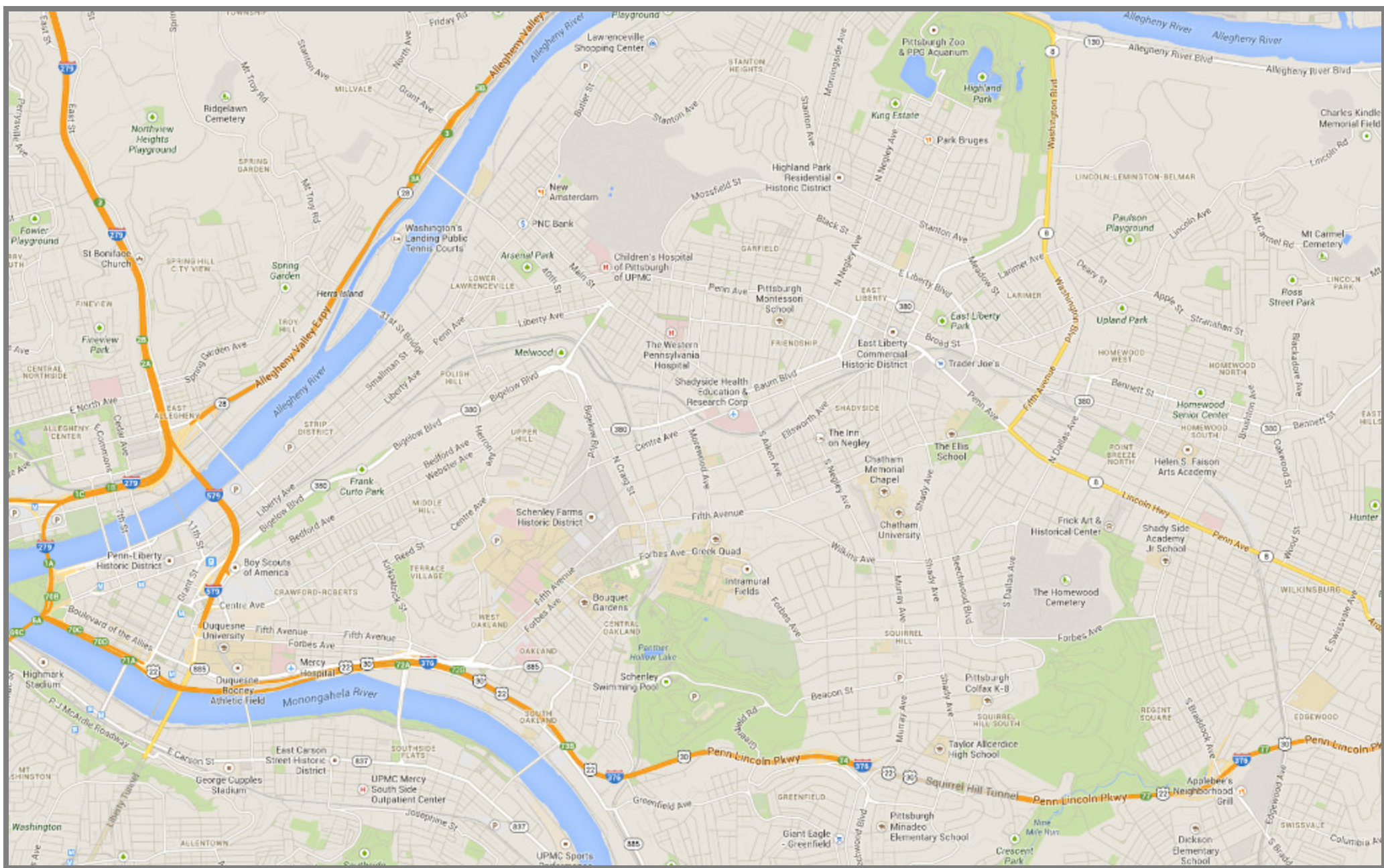


Recall: Systems Thinking



A system is a set of inter-related components that work together in a particular environment to perform whatever functions are required to achieve the system's objective -- Donella Meadows

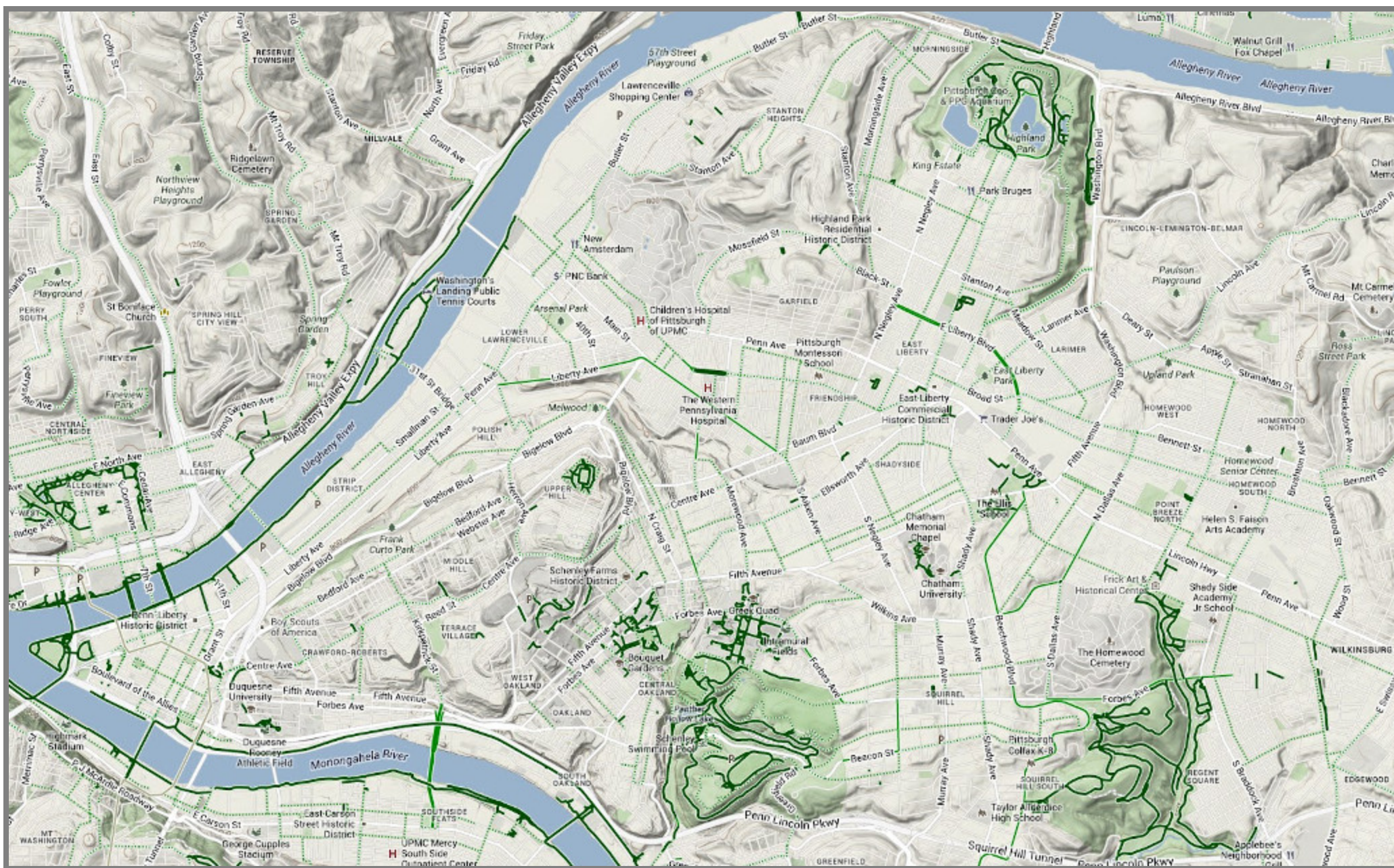
Architectural Modeling and Reasoning



Speaker notes

Map of Pittsburgh. Abstraction for navigation with cars.

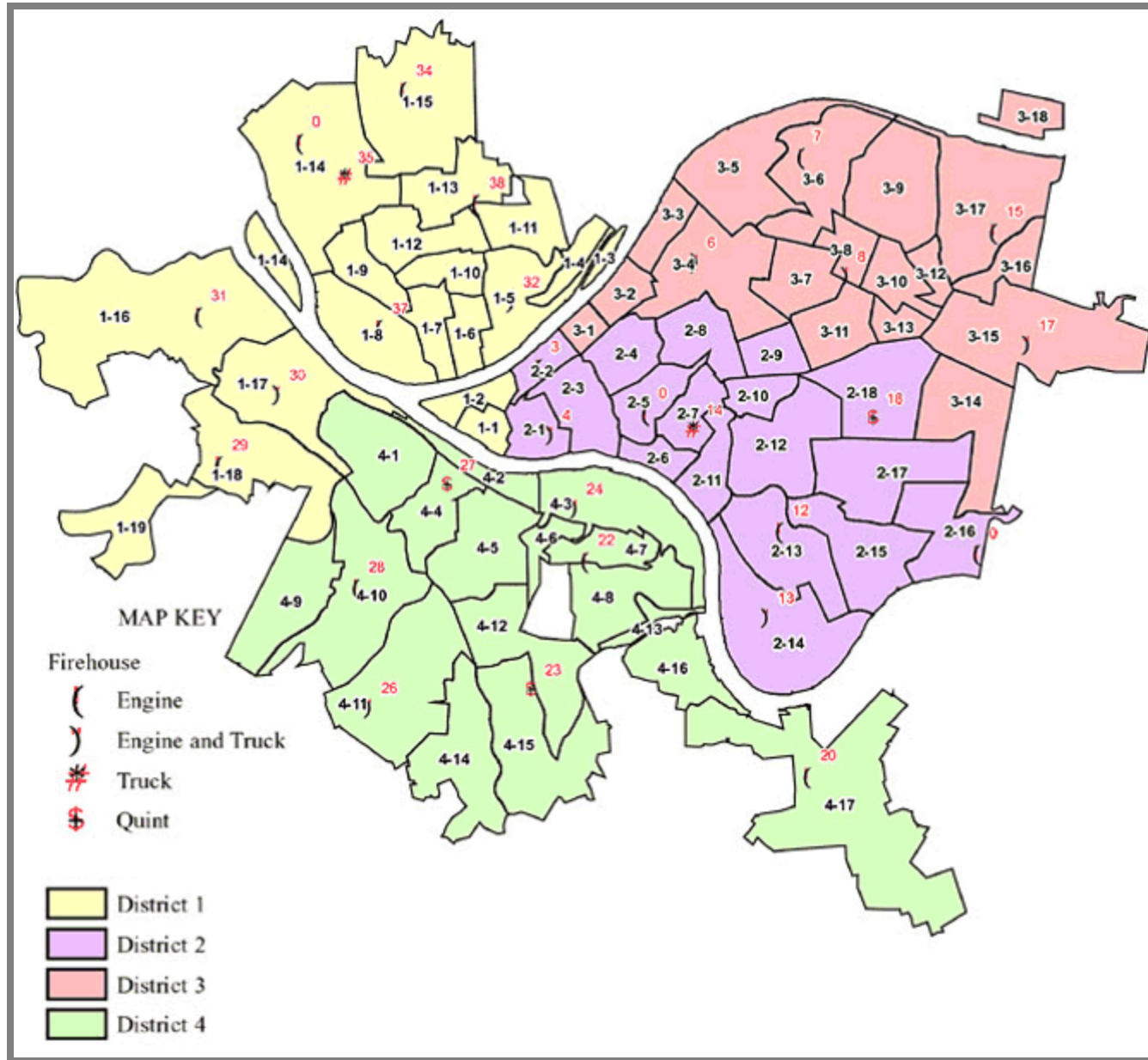




Speaker notes

Cycling map of Pittsburgh. Abstraction for navigation with bikes and walking.





Fire zones of Pittsburgh. Various use cases, e.g., for city planners.



Analysis-Specific Abstractions

All maps were abstractions of the same real-world construct

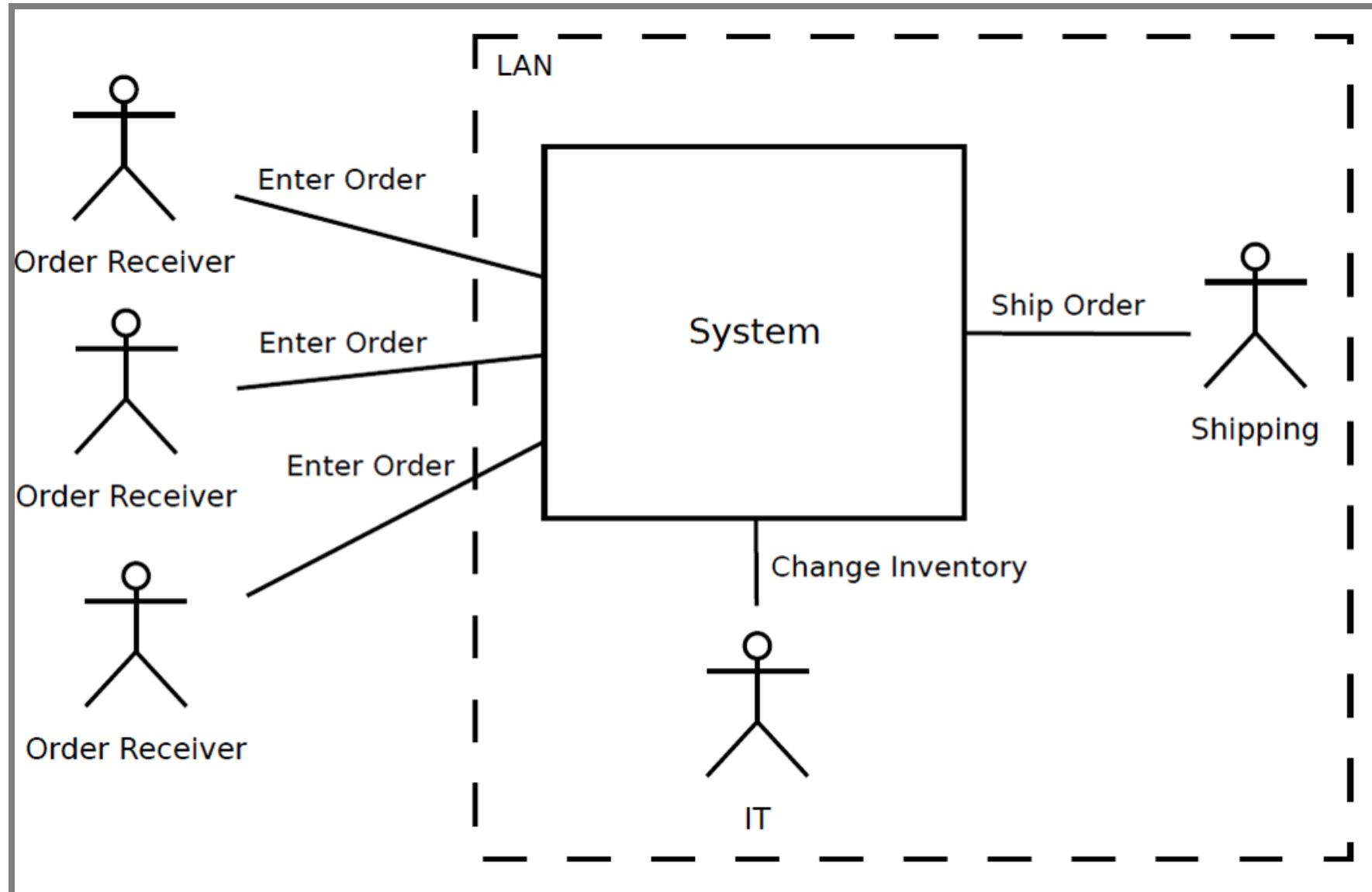
All maps were created with different goals in mind

- Different relevant abstractions
- Different reasoning opportunities

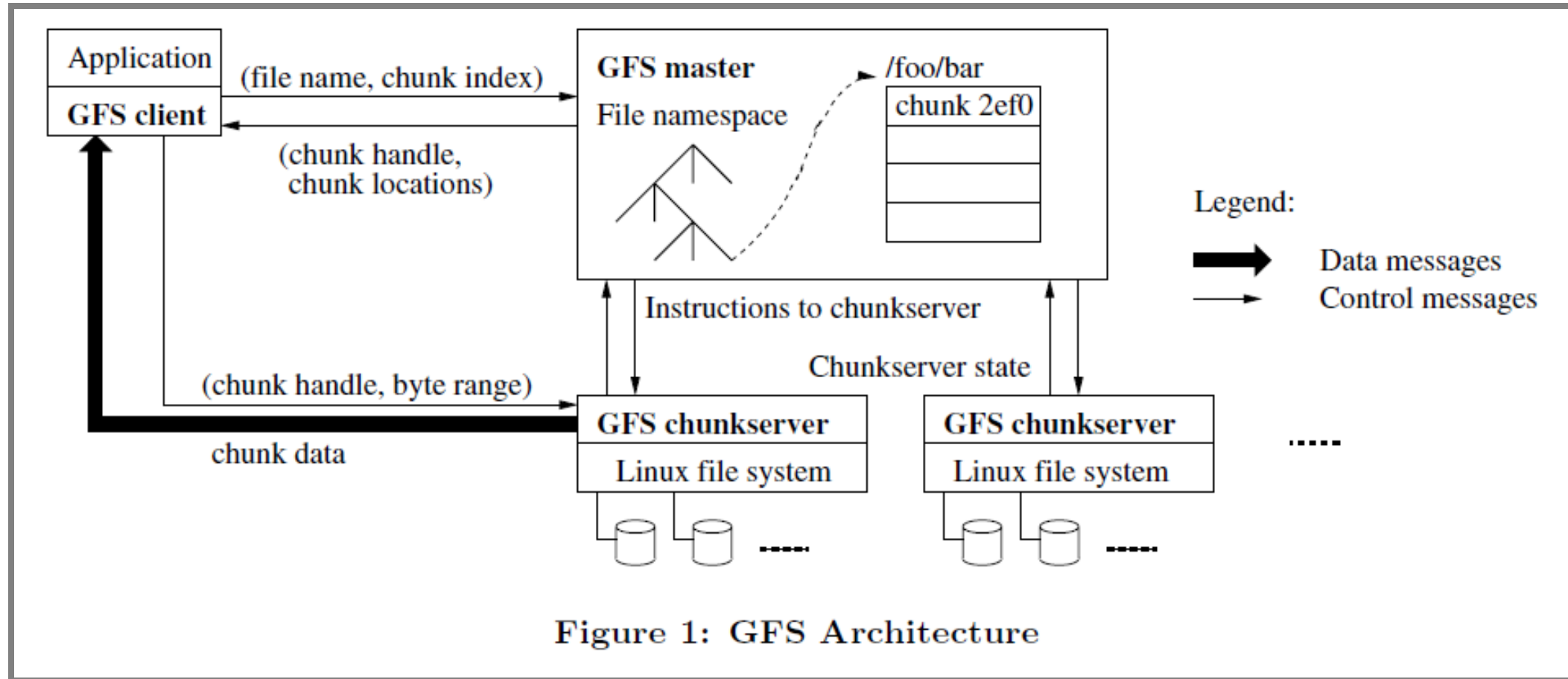
Architectural models are specific system abstractions, for reasoning about specific qualities

No uniform notation

What can we reason about?



What can we reason about?

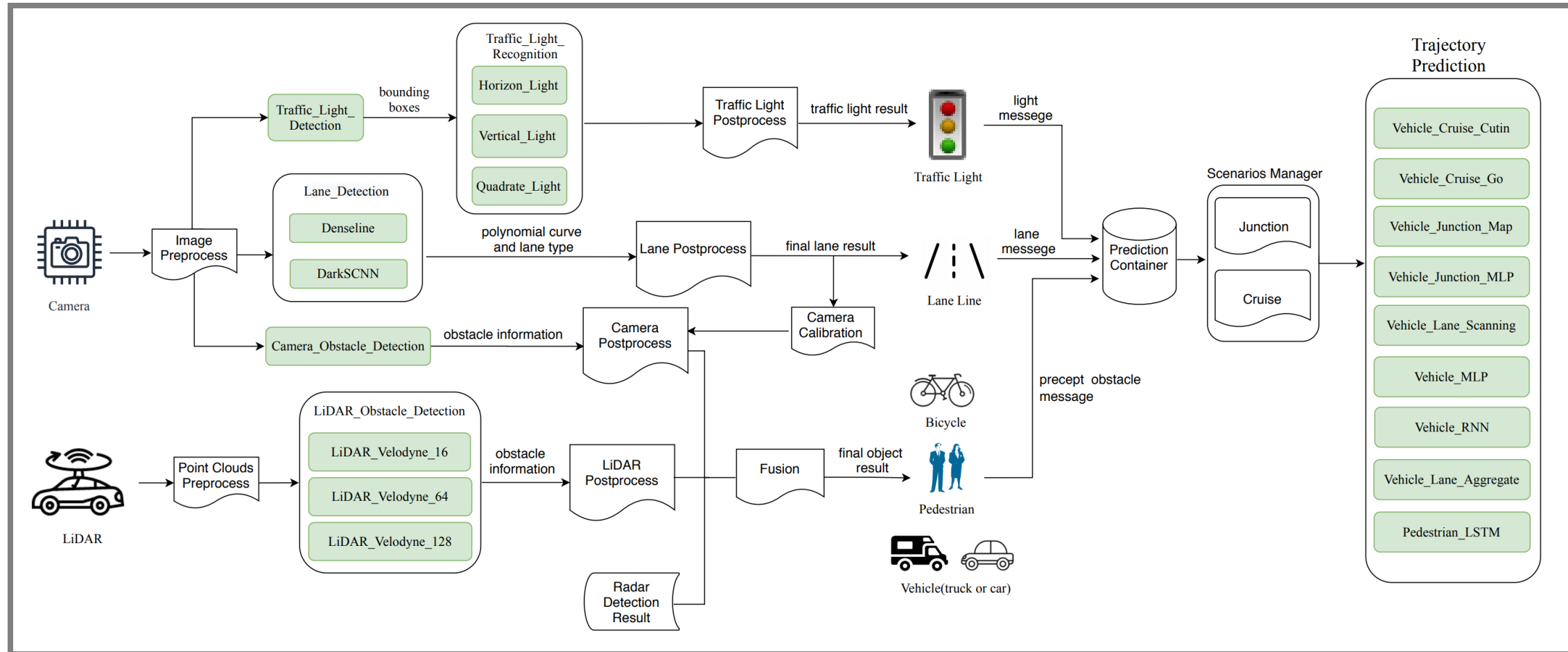


Ghemawat, Sanjay, Howard Gobioff, and Shun-Tak Leung. "The Google file system." ACM SIGOPS operating systems review. Vol. 37. No. 5. ACM, 2003.

Scalability through redundancy and replication; reliability wrt to single points of failure; performance on edges; cost



What can we reason about?



Peng, Zi, Jinqiu Yang, Tse-Hsun Chen, and Lei Ma. "A first look at the integration of machine learning models in complex autonomous driving systems: a case study on Apollo." In Proc. FSE, 2020.

Suggestions for Graphical Notations

Use notation suitable for analysis

Document meaning of boxes and edges in legend

Graphical or textual both okay; whiteboard sketches often sufficient

Formal notations available

Case Study: Augmented Reality Translation



Speaker notes

Image: <https://pixabay.com/photos/nightlife-republic-of-korea-jongno-2162772/>



Case Study: Augmented Reality Translation



Case Study: Augmented Reality Translation



Consider you want to implement an instant translation service similar toGoogle translate, but run it on embedded hardware in glasses as an augmented reality service.



System Qualities of Interest?



Design Decision: Selecting ML Algorithms

What ML algorithms to use and why? Tradeoffs?

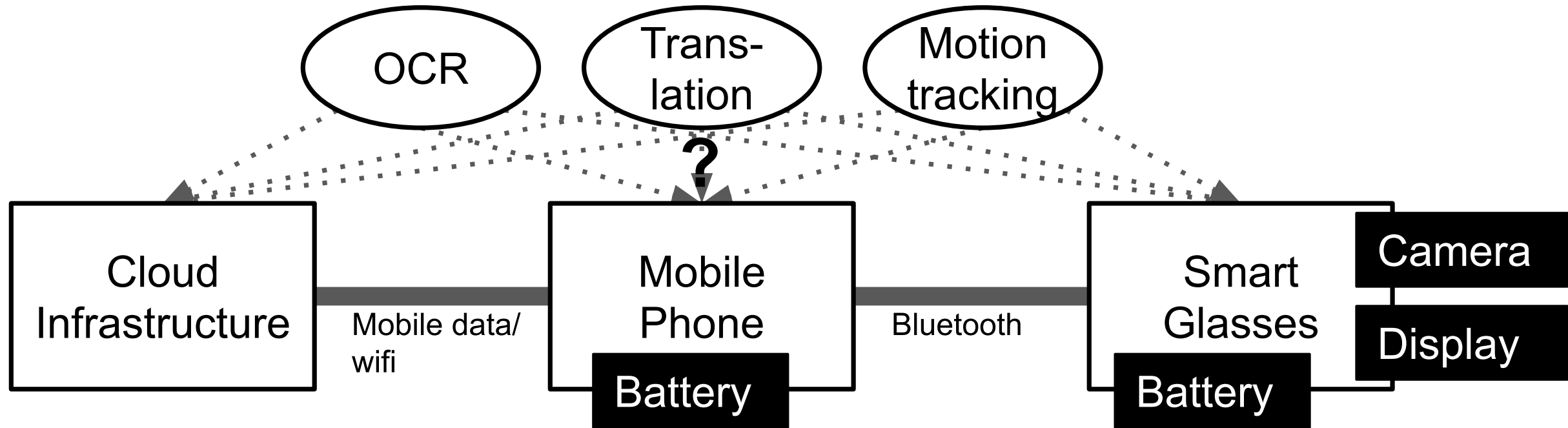


Relate back to previous lecture about AI technique tradeoffs, including for example Accuracy Capabilities (e.g. classification, recommendation, clustering...) Amount of training data needed Inference latency Learning latency; incremental learning? Model size Explainable? Robust?

Design Decision: Where Should the Model Live?

(Deployment Architecture)

Where Should the Models Live?



Cloud? Phone? Glasses?

What qualities are relevant for the decision?

Speaker notes

Trigger initial discussion

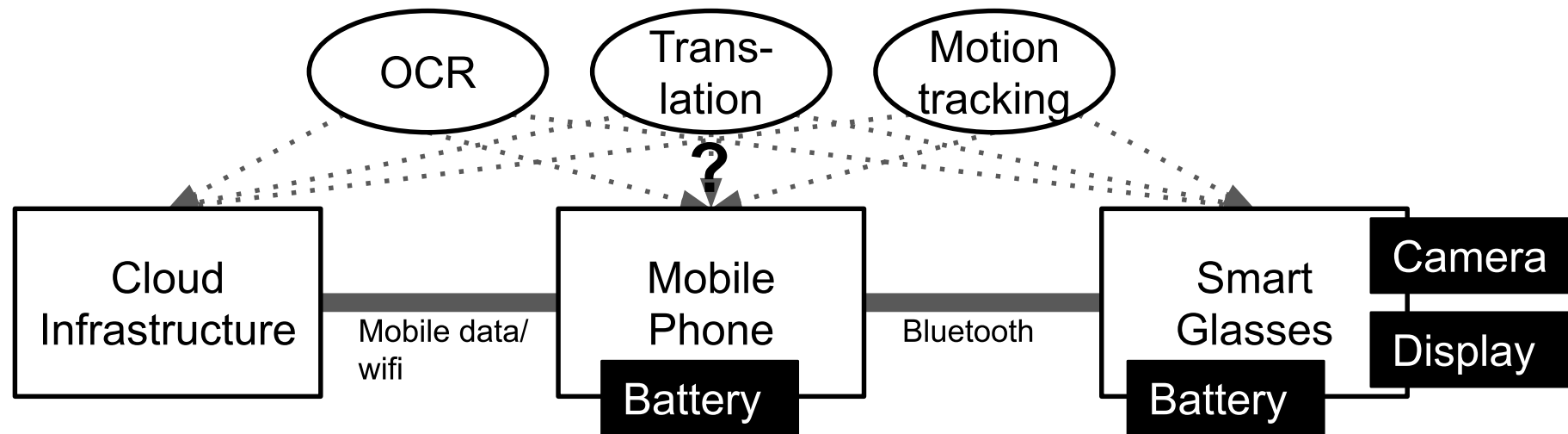


Considerations

- How much data is needed as input for the model?
- How much output data is produced by the model?
- How fast/energy consuming is model execution?
- What latency is needed for the application?
- How big is the model? How often does it need to be updated?
- Cost of operating the model? (distribution + execution)
- Opportunities for telemetry?
- What happens if users are offline?

Breakout: Latency and Bandwidth Analysis

1. Estimate latency and bandwidth requirements between components
2. Discuss tradeoffs among different deployment models



As a group, post in #lecture tagging group members:

Identify at least OCR and Translation service as two AI components in a larger system. Discuss which system components are worth modeling (e.g., rendering, database, support forum). Discuss how to get good estimates for latency and bandwidth.

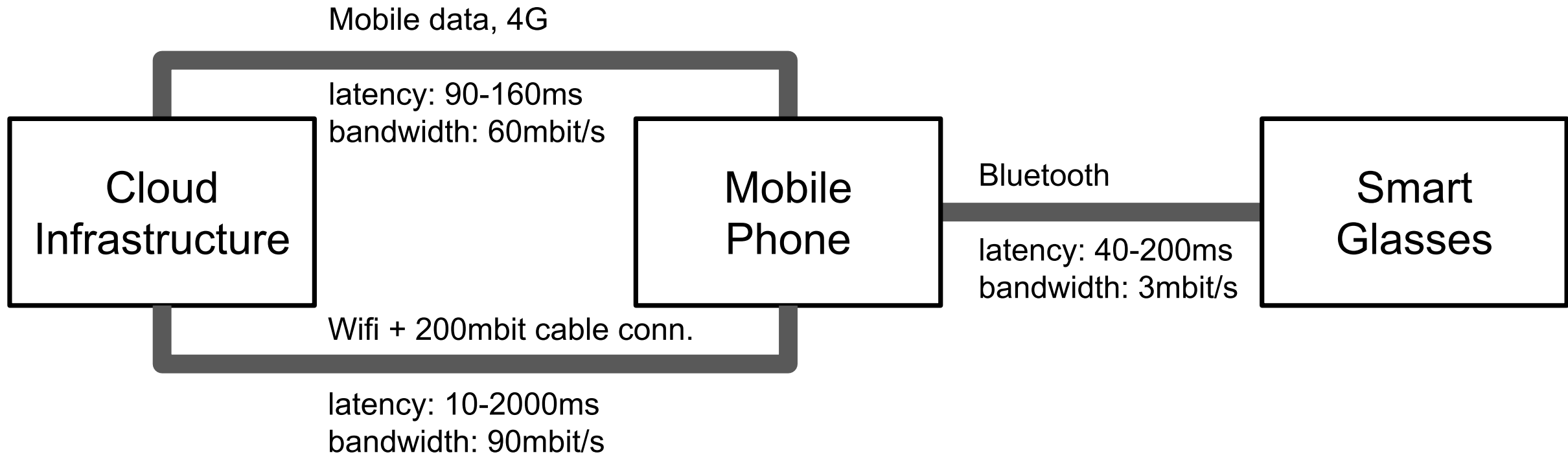
Some data: 200ms latency is noticable as speech pause; 20ms is perceivable as video delay, 10ms as haptic delay; 5ms referenced as cybersickness threshold for virtual reality 20ms latency might be acceptable

bluetooth latency around 40ms to 200ms

bluetooth bandwidth up to 3mbit, wifi 54mbit, video stream depending on quality 4 to 10mbit for low to medium quality

google glasses had 5 megapixel camera, 640x360 pixel screen, 1 or 2gb ram, 16gb storage





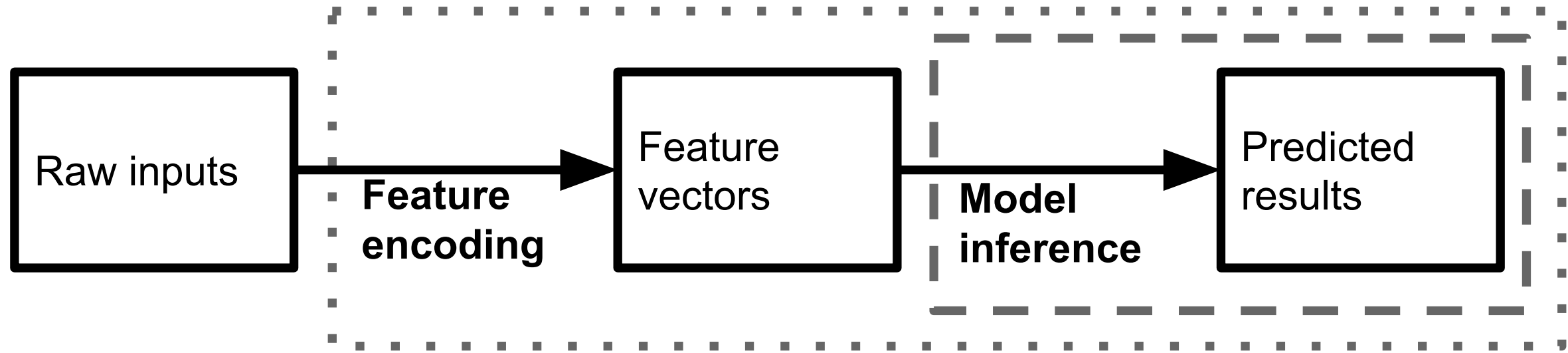
From the Reading: When would one use the following designs?

- Static intelligence in the product
 - Client-side intelligence (user-facing devices)
 - Server-centric intelligence
 - Back-end cached intelligence
 - Hybrid models
-
- Consider: Offline use, inference latency, model updates, application updates, operating cost, scalability, protecting intellectual property

From the reading:

- Static intelligence in the product
 - difficult to update
 - good execution latency
 - cheap operation
 - offline operation
 - no telemetry to evaluate and improve
- Client-side intelligence
 - updates costly/slow, out of sync problems
 - complexity in clients
 - offline operation, low execution latency
- Server-centric intelligence
 - latency in model execution (remote calls)
 - easy to update and experiment
 - operation cost
 - no offline operation
- Back-end cached intelligence
 - precomputed common results
 - fast execution, partial offline
 - saves bandwidth, complicated updates
- Hybrid models

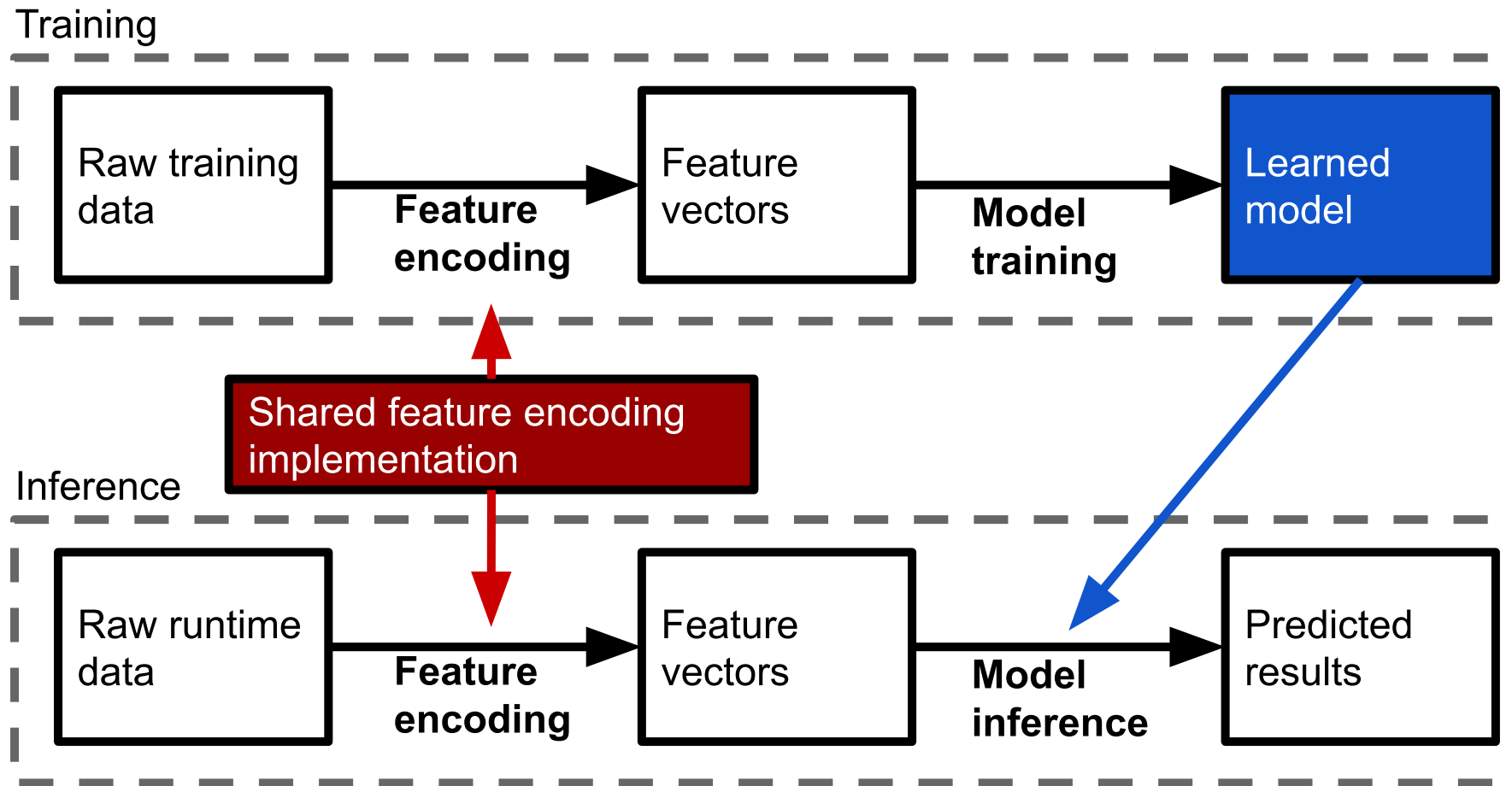
Where Should Feature Encoding Happen?



Should feature encoding happen server-side or client-side? Tradeoffs?

When thinking of model inference as a component within a system, feature encoding can happen with the model-inference component or can be the responsibility of the client. That is, the client either provides the raw inputs (e.g., image files; dotted box in the figure above) to the inference service or the client is responsible for computing features and provides the feature vector to the inference service (dashed box). Feature encoding and model inference could even be two separate services that are called by the client in sequence. Which alternative is preferable is a design decision that may depend on a number of factors, for example, whether and how the feature vectors are stored in the system, how expensive computing the feature encoding is, how often feature encoding changes, how many models use the same feature encoding, and so forth. For instance, in our stock photo example, having feature encoding being part of the inference service is convenient for clients and makes it easy to update the model without changing clients, but we would have to send the entire image over the network instead of just the much smaller feature vector for the reduced 300 x 300 pixels.

Reusing Feature Engineering Code



≡ Avoid *training-serving skew*

The Feature Store Pattern

- Central place to store, version, and describe feature engineering code
- Can be reused across projects
- Possible caching of expensive features

Many open source and commercial offerings, e.g., Feast, Tecton, AWS SageMaker Feature Store

Tecton Feature Store



More Considerations for Deployment Decisions

Coupling of ML pipeline parts

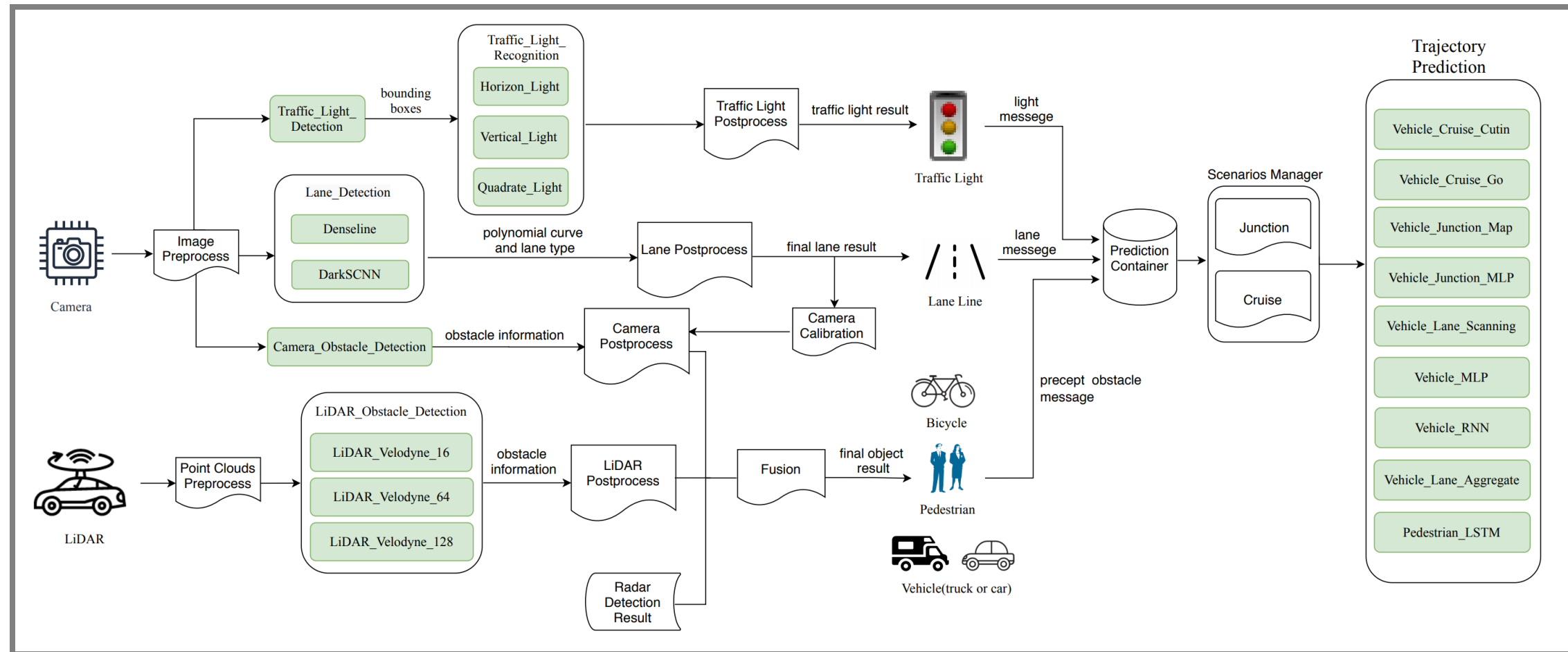
Coupling with other parts of the system

Ability for different developers and analysts to collaborate

Support online experiments

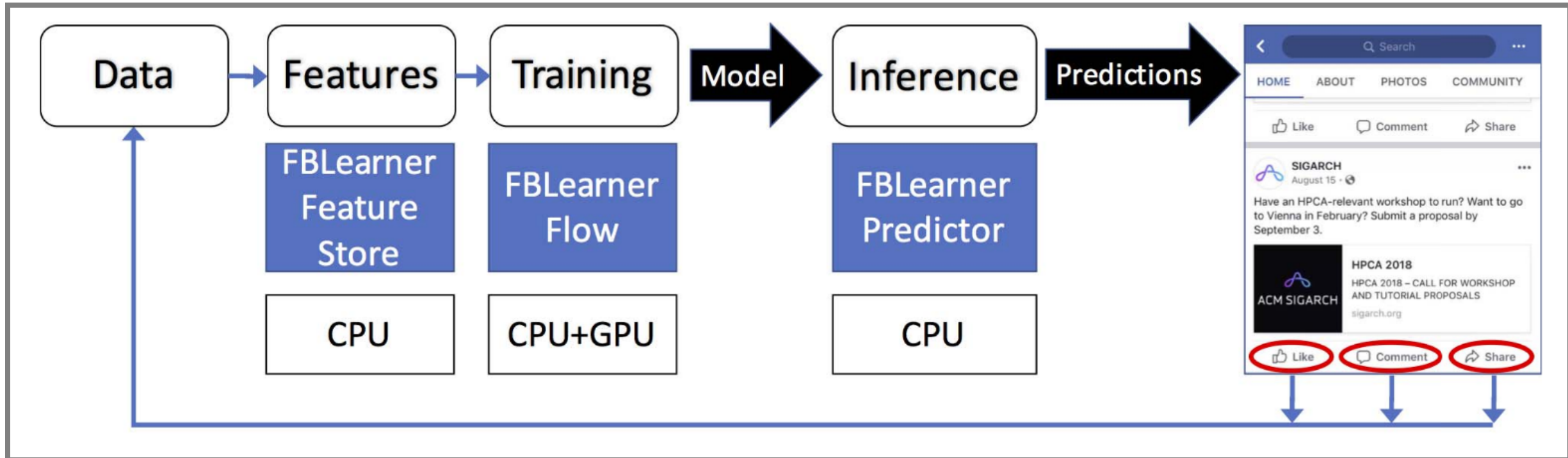
Ability to monitor

Real-Time Serving; Many Models



Peng, Zi, Jinqiu Yang, Tse-Hsun Chen, and Lei Ma. "A first look at the integration of machine learning models in complex autonomous driving systems: a case study on Apollo." In Proc. FSE. 2020.

Infrastructure Planning (Facebook Examp.)



Hazelwood, Kim, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy et al. "Applied machine learning at facebook: A datacenter infrastructure perspective." In Int'l Symp. High Performance Computer Architecture. IEEE, 2018.

Operational Robustness

Redundancy for availability?

Load balancer for scalability?

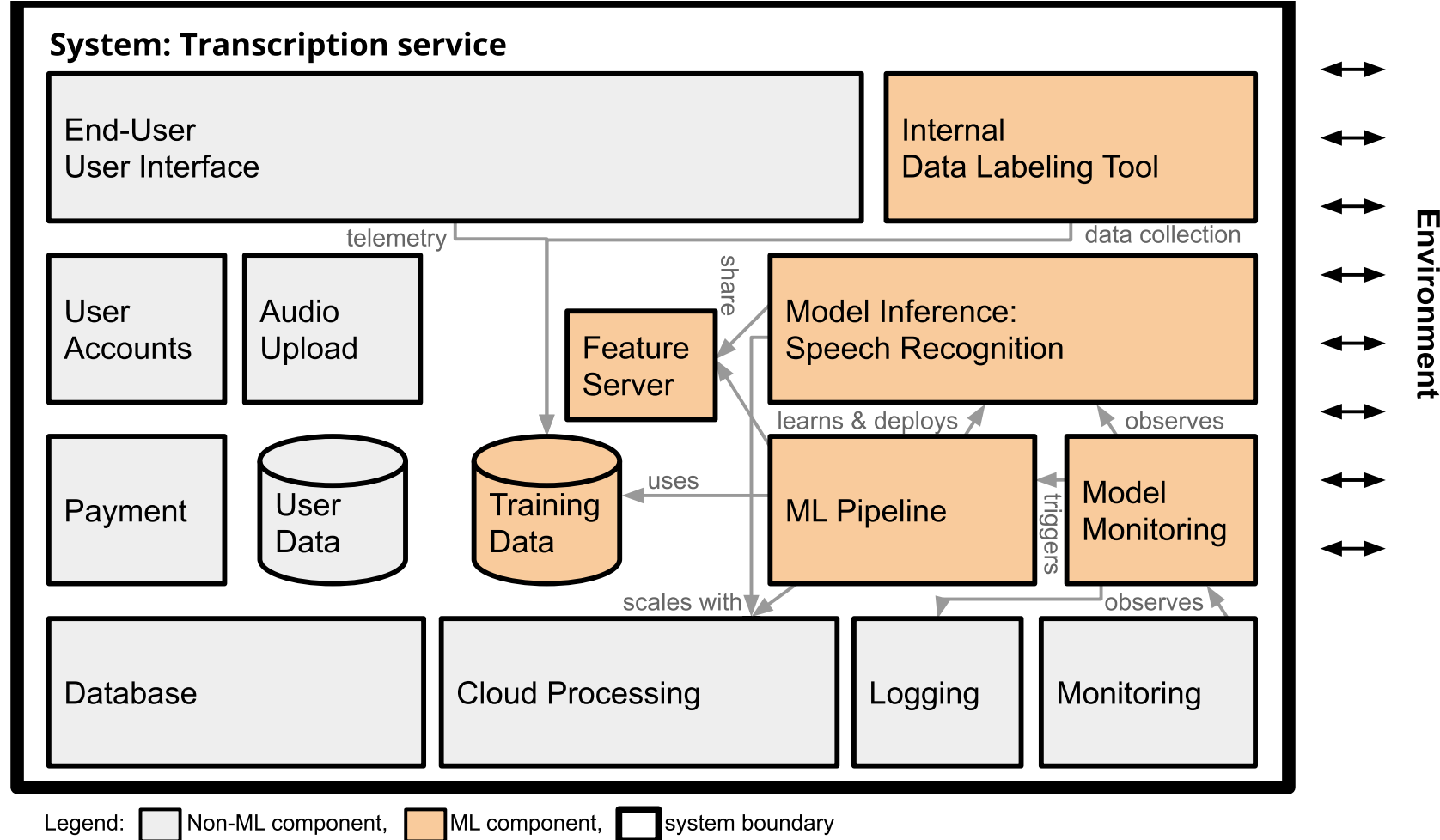
Can mistakes be isolated?

- Local error handling?
- Telemetry to isolate errors to component?

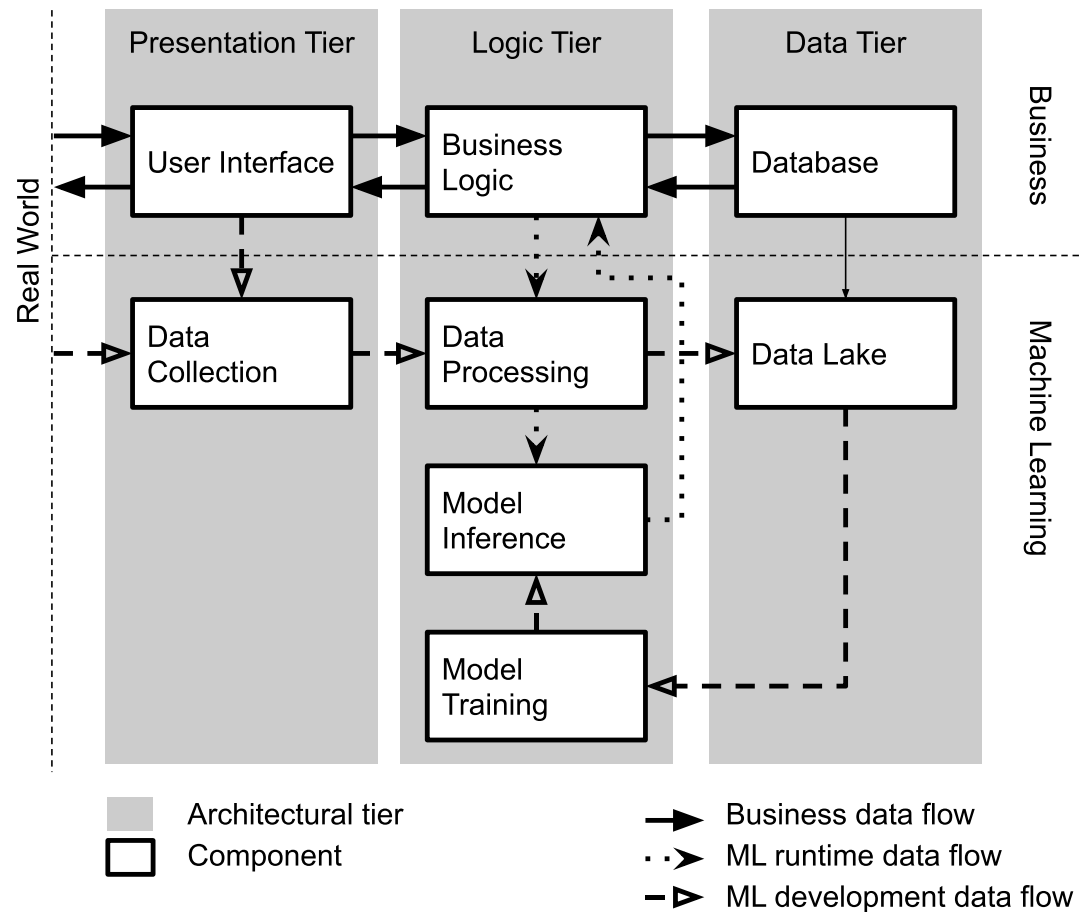
Logging and log analysis for what qualities?

Integrating Models into a System

Recall: Inference is a Component within a System



Separating Models and Business Logic



Based on: Yokoyama, Haruki. "Machine learning system architectural pattern for improving operational stability." In Int'l Conf. Software Architecture Companion, pp. 267-274. IEEE, 2019.

Separating Models and Business Logic

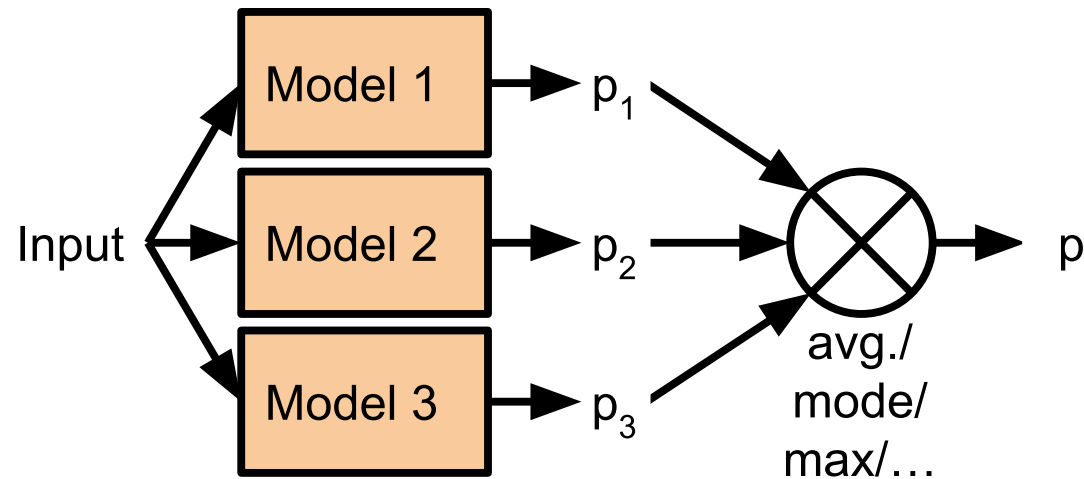
Clearly divide responsibilities

Allows largely independent and parallel work, assuming stable interfaces

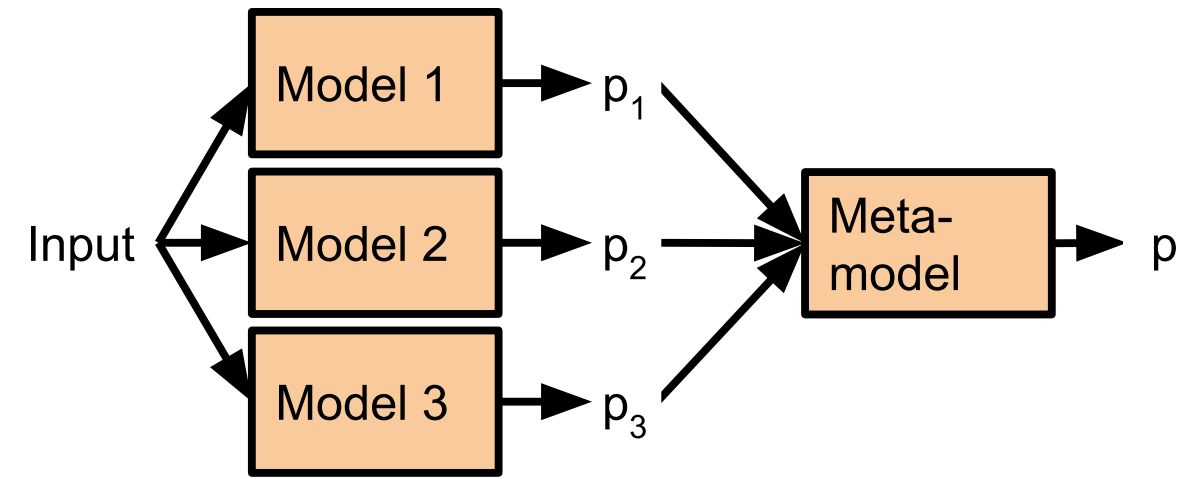
Plan location of non-ML safeguards and other processing logic

Composing Models: Ensemble and metamodels

Ensemble

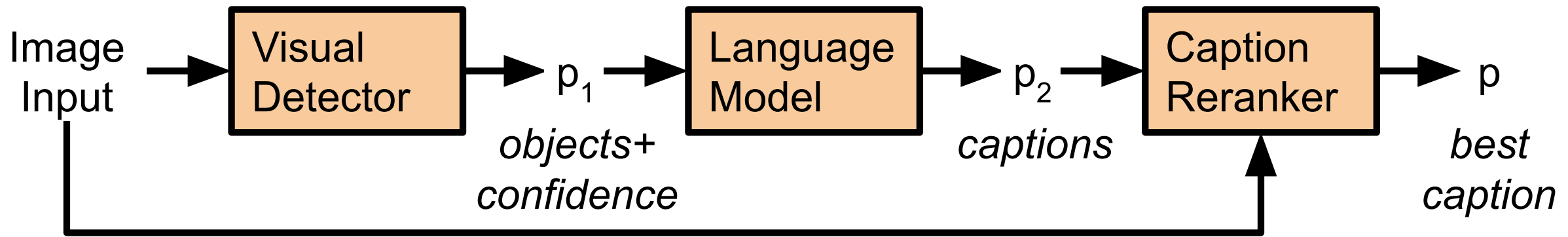


Metamodel / model stacking

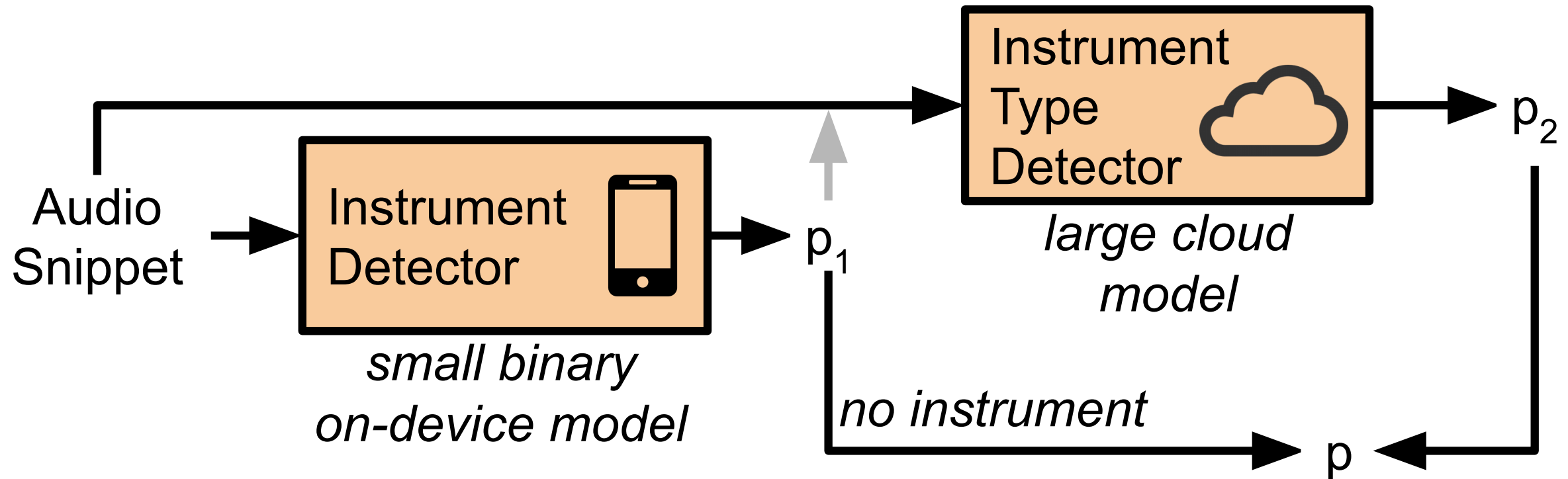


Legend:  machine-learned model,  non-ML aggregation function, p prediction

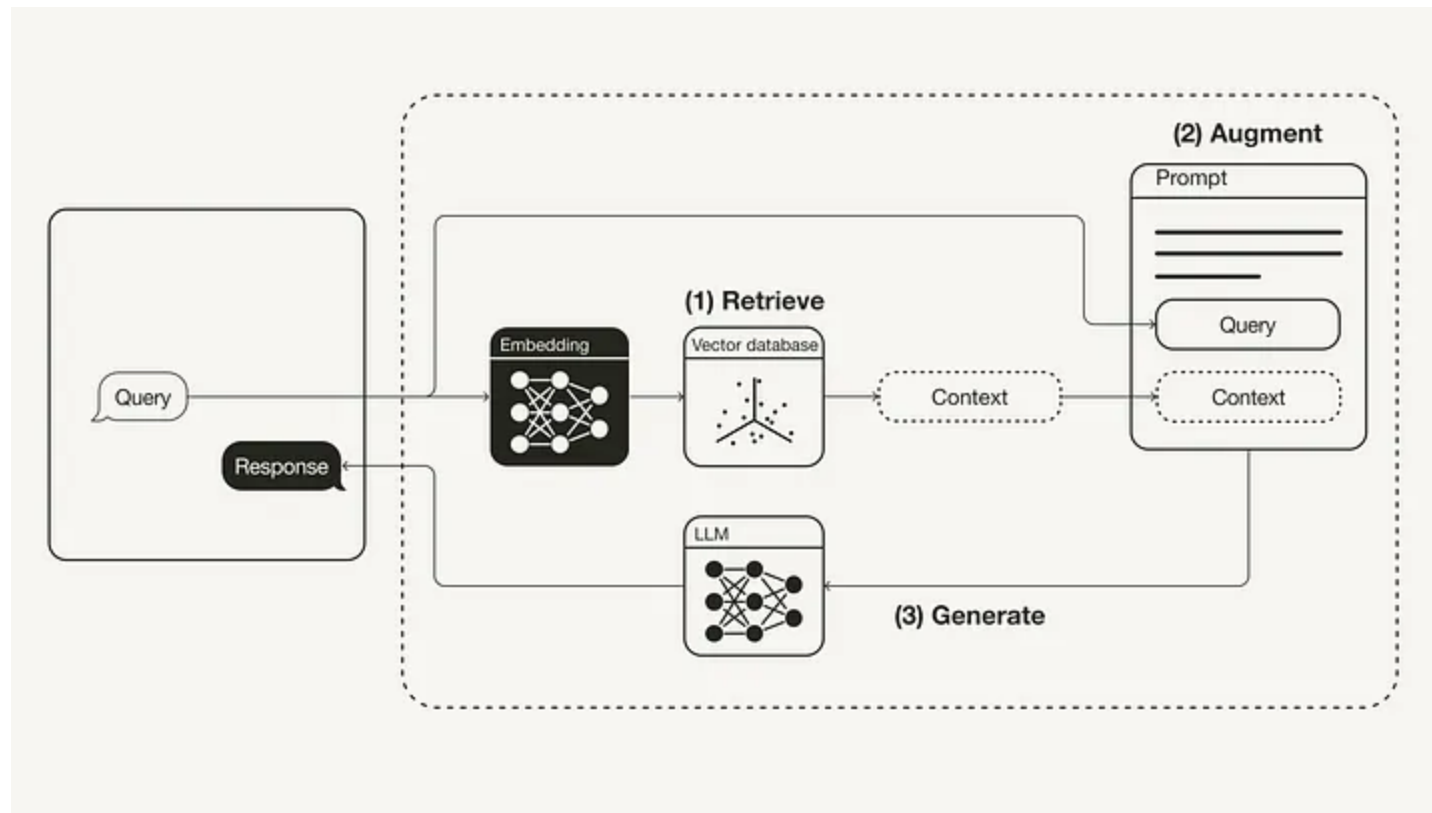
Composing Models: Decomposing the problem, sequential



Composing Models: Cascade/two-phase prediction



Composing Models: Retrieval Augmented Generation (RAG)



Documenting Model Inference Interfaces

Why Documentation

Model inference between teams:

- Data scientists developing the model
- Other data scientists using the model, evolving the model
- Software engineers integrating the model as a component
- Operators managing model deployment

Will this model work for my problem?

What problems to anticipate?

Classic API Documentation

```
/**  
 * compute deductions based on provided adjusted  
 * gross income and expenses in customer data.  
 *  
 * see tax code 26 U.S. Code A.1.B, PART VI  
 */  
float computeDeductions(float agi, Expenses expenses);
```

What to document for models?



Documenting Input/Output Types for Inference Components

```
{  
  "mid": string,  
  "languageCode": string,  
  "name": string,  
  "score": number,  
  "boundingPoly": {  
    object (BoundingPoly)  
  }  
}
```

From Google's public [object detection API](#).

Documentation beyond I/O Types

Intended use cases, model capabilities and limitations

Supported target distribution (vs preconditions)

Accuracy (various measures), incl. slices, fairness

Latency, throughput, availability (service level agreements)

Model qualities such as explainability, robustness, calibration

Ethical considerations (fairness, safety, security, privacy)

Example for OCR model? How would you describe these?

Model Cards

- Proposal and template for documentation from Google
 - Intended use, out-of-scope use
 - Training and evaluation data
 - Considered demographic factors
 - Accuracy evaluations
 - Ethical considerations
- 1-2 page summary
- Focused on fairness
- Widely discussed, but not frequently adopted

Mitchell, Margaret, et al. "[Model cards for model reporting](#)." In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019.

Model Card - Toxicity in Text

Model Details

- The TOXICITY classifier provided by Perspective API [32], trained to predict the likelihood that a comment will be perceived as toxic.
- Convolutional Neural Network.
- Developed by Jigsaw in 2017.

Intended Use

- Intended to be used for a wide range of use cases such as supporting human moderation and providing feedback to comment authors.
- Not intended for fully automated moderation.
- Not intended to make judgments about specific individuals.

Factors

- Identity terms referencing frequently attacked groups, focusing on sexual orientation, gender identity, and race.

Metrics

- Pinned AUC, as presented in [11], which measures threshold-agnostic separability of toxic and non-toxic comments for each group, within the context of a background distribution of other groups.

Ethical Considerations

- Following [31], the Perspective API uses a set of values to guide their work. These values are Community, Transparency, Inclusivity, Privacy, and Topic-neutrality. Because of privacy considerations, the model does not take into account user history when making judgments about toxicity.

Quantitative Analyses

Training Data

- Proprietary from Perspective API. Following details in [11] and [32], this includes comments from a online forums such as Wikipedia and New York Times, with crowdsourced labels of whether the comment is “toxic”.
- “Toxic” is defined as “a rude, disrespectful, or unreasonable comment that is likely to make you leave a discussion.”

Evaluation Data

- A synthetic test set generated using a template-based approach, as suggested in [11], where identity terms are swapped into a variety of template sentences.
- Synthetic data is valuable here because [11] shows that real data often has disproportionate amounts of toxicity directed at specific groups. Synthetic data ensures that we evaluate on data that represents both toxic and non-toxic statements referencing a variety of groups.

Caveats and Recommendations

- Synthetic test data covers only a small set of very specific comments. While these are designed to be representative of common use cases and concerns, it is not comprehensive.

The figure consists of four bar charts arranged in a 2x2 grid, showing the Pinned AUC (Area Under the Curve) for different groups. The top row shows 'Pinned AUC by Unitary Groups' for Version 1 and Version 5. The bottom row shows 'Pinned AUC by Intersectional Groups' for Version 1 and Version 5. The y-axis for all charts ranges from 0.80 to 1.00. The x-axis for the unitary charts lists groups: lesbian, gpt, queer, straight, bisexual, heterosexual, cis, trans, military, black, and white. The intersectional charts show combinations of these groups, such as black-lesbian, white-lesbian, etc. In the unitary charts, AUC is generally high, around 0.90-0.95. In the intersectional charts, AUC is generally lower, around 0.85-0.90, and shows more variation across groups.

Group	AUC
lesbian	0.90
gpt	0.85
queer	0.95
straight	0.95
bisexual	0.95
heterosexual	0.85
cis	0.95
trans	0.95
military	0.95
black	0.90
white	0.90

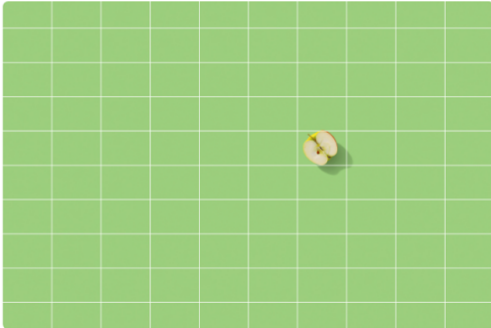
Group	AUC
lesbian	0.95
gpt	0.95
queer	0.95
straight	0.95
bisexual	0.95
heterosexual	0.95
cis	0.95
trans	0.95
military	0.95
black	0.95
white	0.95

Group	AUC
black-lesbian	0.85
white-lesbian	0.90
black-gpt	0.85
white-gpt	0.90
black-queer	0.90
white-queer	0.95
black-straight	0.95
white-straight	0.95
black-bisexual	0.90
white-bisexual	0.95
black-heterosexual	0.85
white-heterosexual	0.90
black-cis	0.95
white-cis	0.95
black-trans	0.90
white-trans	0.95
black-military	0.95
white-military	0.95

Group	AUC
black-lesbian	0.95
white-lesbian	0.95
black-gpt	0.95
white-gpt	0.95
black-queer	0.95
white-queer	0.95
black-straight	0.95
white-straight	0.95
black-bisexual	0.95
white-bisexual	0.95
black-heterosexual	0.95
white-heterosexual	0.95
black-cis	0.95
white-cis	0.95
black-trans	0.95
white-trans	0.95
black-military	0.95
white-military	0.95

Limitations

The following factors may degrade the model's performance.



Object size: Object size must be at least 1% of the image area to be detected.



"Things" vs "stuff": Model was designed to detect discrete objects with clearly discernible shapes ("things"), not a group of overlapping objects or background clutter ("stuff").



Lighting: Poor or harsh, high-contrast illumination (e.g. nighttime, back-lit, side-lit) may degrade model performance.



Occlusion or clutter: Partially obstructed or truncated objects may not be detected. For example, a shirt underneath a jacket, or where less than 25% of an object is visible in the image.



Camera positioning and lens type: Camera angle and positioning (e.g. oblique angles, long-distance), and lens type (e.g. fisheye) may impact model performance.



Blur or noise: Blurry objects, rapid movement between frames, or encoding/decoding noise may degrade model performance.

FactSheets

Proposal and template for documentation from IBM; intended to communicate intended qualities and assurances

Longer list of criteria, including

- Service intention, intended use
- Technical description
- Target distribution
- Own and third-party evaluation results
- Safety and fairness considerations, explainability
- Preparation for drift and evolution
- Security, lineage and versioning

Arnold, Matthew, et al. "[FactSheets: Increasing trust in AI services through supplier's declarations of conformity](#)." *IBM Journal of Research and Development* 63, no. 4/5 (2019): 6-1.

Recall: Correctness vs Fit

Without a clear specification a model is difficult to document

Need documentation to allow evaluation for *fit*

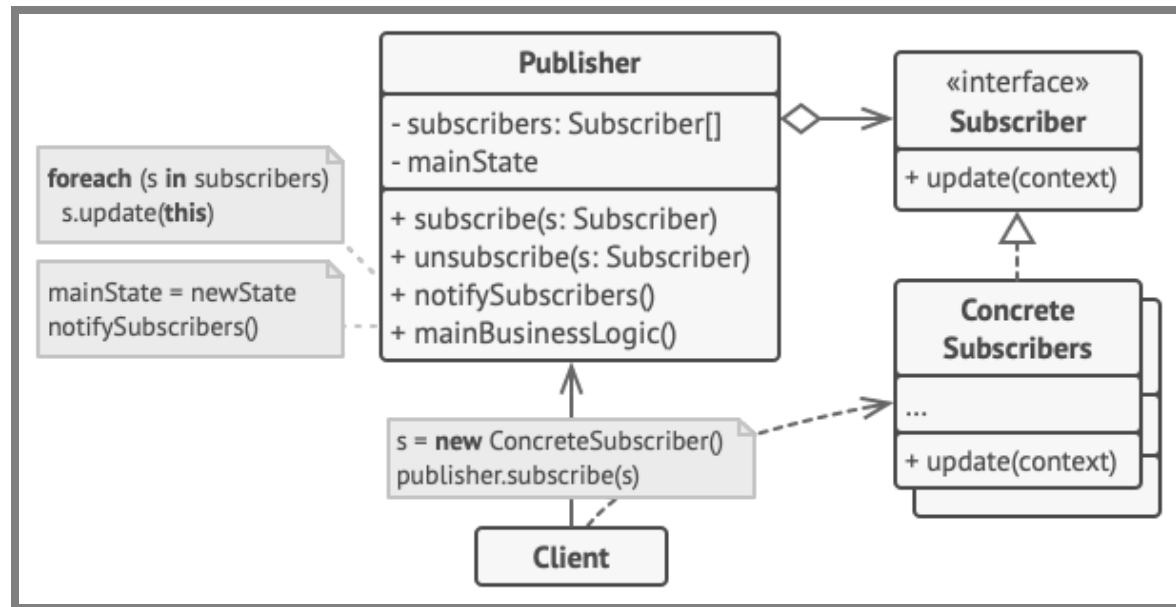
Description of *target distribution* is a key challenge

Design Patterns for AI Enabled Systems

(no standardization, yet)

Design Patterns are Codified Design Knowl.

Vocabulary of design problems and solutions



Example: *Observer pattern* object-oriented design pattern describes a
≡ solution how objects can be notified when another object changes

Common System Structures

Client-server architecture

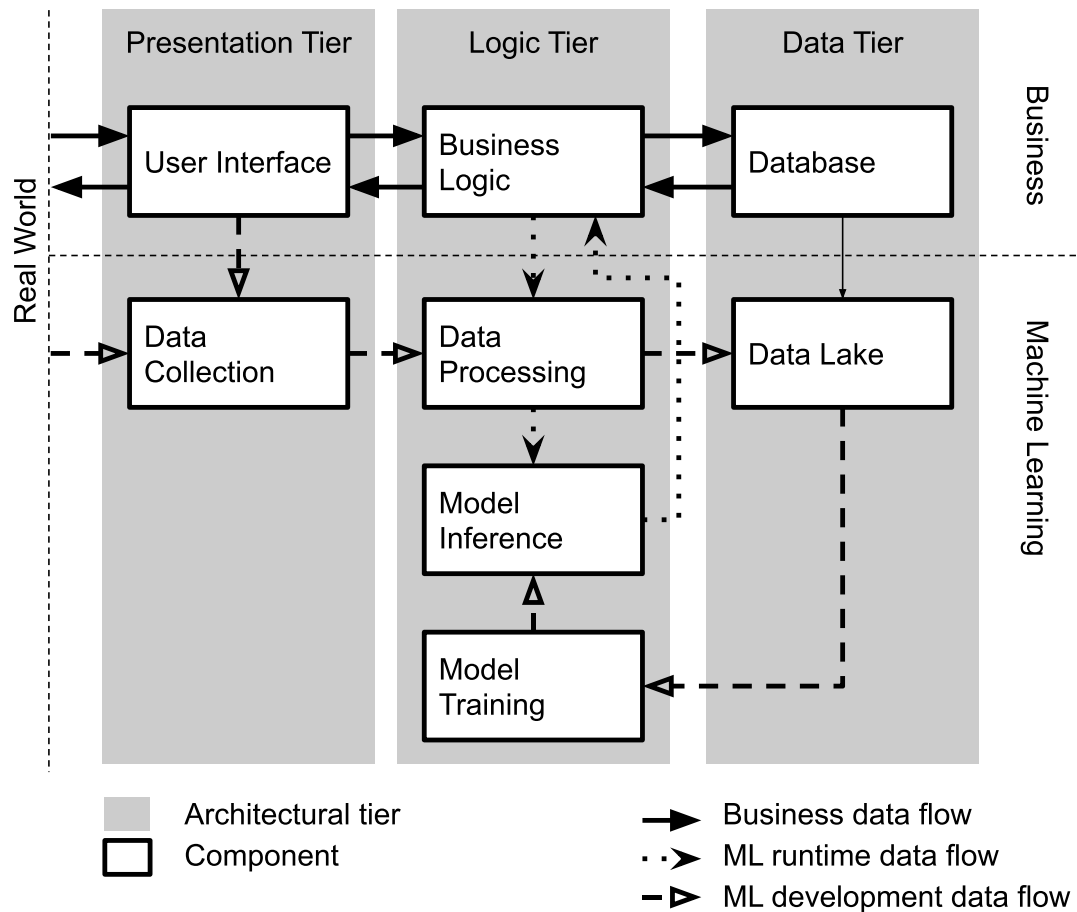
Multi-tier architecture

Service-oriented architecture and microservices

Event-based architecture

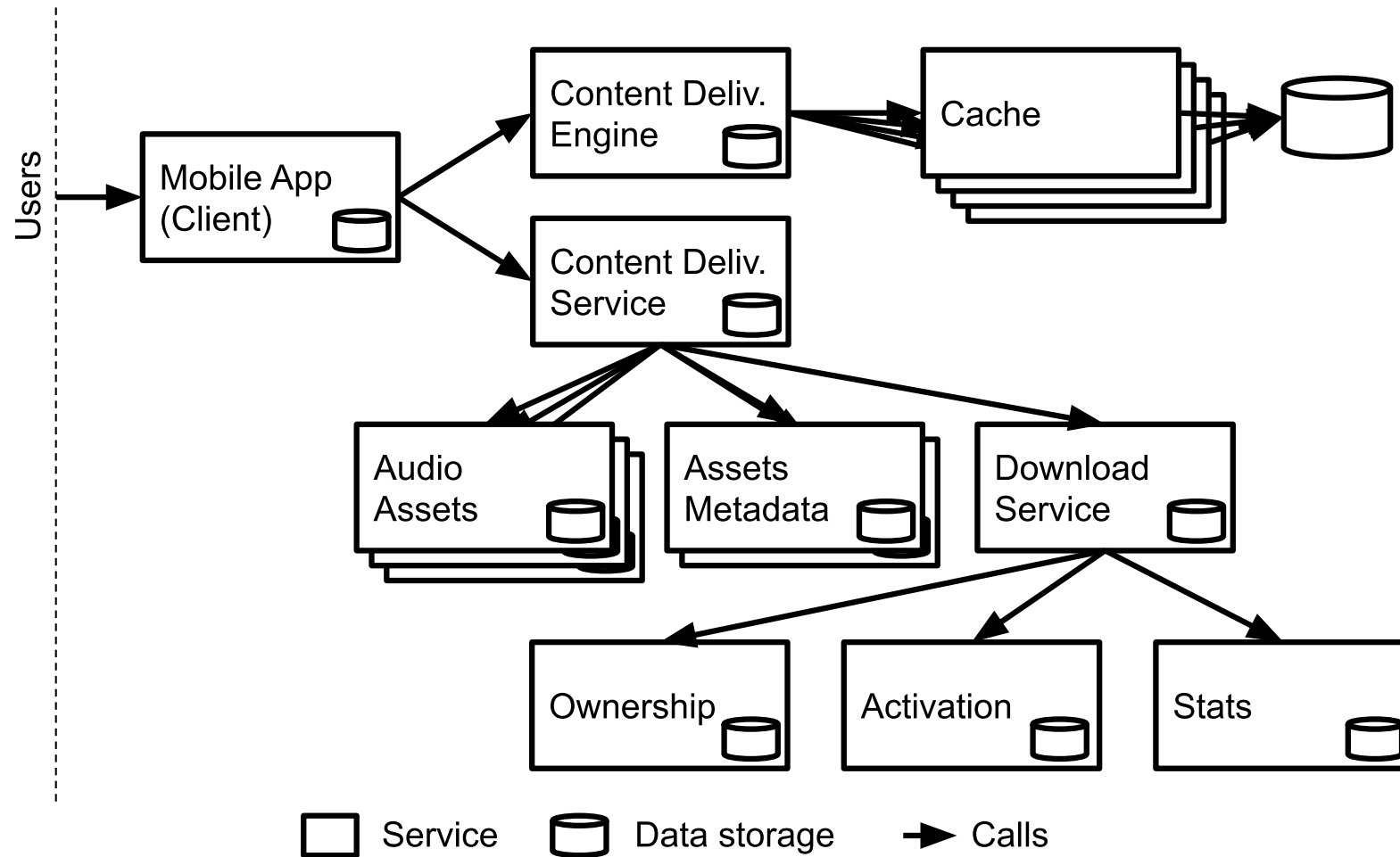
Data-flow architecture

Multi-Tier Architecture



Based on: Yokoyama, Haruki. "Machine learning system architectural pattern for improving operational stability." In Int'l Conf. Software Architecture Companion, pp. 267-274. IEEE, 2019.

Microservices



≡ (more later)

Patterns for ML-Enabled Systems

- Stateless/serverless Serving Function Pattern
- Feature-Store Pattern
- Batched/precomputed serving pattern
- Two-phase prediction pattern
- Batch Serving Pattern
- Decouple-training-from-serving pattern

Anti-Patterns

- Big Ass Script Architecture
- Dead Experimental Code Paths
- Glue code
- Multiple Language Smell
- Pipeline Jungles
- Plain-Old Datatype Smell
- Undeclared Consumers

See also: Washizaki, Hironori, Hiromu Uchida, Foutse Khomh, and Yann-Gaël Guéhéneuc. "[Machine Learning Architecture and Design Patterns](#)." Draft, 2019; Sculley, et al. "[Hidden technical debt in machine learning systems](#)." In NeurIPS, 2015.

Summary

Model deployment seems easy, but involves many design decisions

- What models to use?
- Where to deploy?
- How to design feature encoding and feature engineering?
- How to compose with other components?
- How to document?
- How to collect telemetry?

Problem-specific modeling and analysis: Gather estimates, consider design alternatives, make tradeoffs explicit

Codifying design knowledge as patterns

Further Readings

- Lakshmanan, Valliappa, Sara Robinson, and Michael Munn. Machine learning design patterns. O'Reilly Media, 2020.
- Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. “Model cards for model reporting.” In Proceedings of the conference on fairness, accountability, and transparency, pp. 220–229. 2019.
- Arnold, Matthew, Rachel KE Bellamy, Michael Hind, Stephanie Houde, Sameep Mehta, Aleksandra Mojsilović, Ravi Nair, Karthikeyan Natesan Ramamurthy, Darrell Reimer, Alexandra Olteanu, David Piorkowski, Jason Tsay, and Kush R. Varshney. “FactSheets: Increasing trust in AI services through supplier’s declarations of conformity.” IBM Journal of Research and Development 63, no. 4/5 (2019): 6–1.
- Yokoyama, Haruki. “Machine learning system architectural pattern for improving operational stability.” In 2019 IEEE International Conference on Software Architecture Companion (ICSA-C), pp. 267–274. IEEE, 2019.

